

Explicații algoritmi și pseudo cod

Problema rucsacului

- Avem n obiecte, fiecare obiect având o valoare (v) și o greutate (w). Avem un rucsac care are o greutate maximă (W) pe care o admite în el.
- Problema presupune căutarea valorii maxime posibile ($v_1+v_2+...+v_n$) cât timp greutatea adunată a obiectelor introduse în ghiozdan ($w_1+w_2+...+w_n$) este mai mică decât W .
- Reprezentarea unei soluții este de forma: $[X_1, X_2, ..., X_n]$, $i \in \{0, 1\}$
- $X_i=0 \rightarrow$ obiectul nu se află în ghiozdan, $X_i=1 \rightarrow$ obiectul se află în ghiozdan
- Pentru generarea unei soluții aleatoare am o funcție cu un parametru, lungimea listei care trebuie generate, și returnează lista generată:

```
def generate(len: int):  
    random_list = []  
    for _ in range(len):  
        random_list.append(random.randint(a=0, b=1))  
    # print(random_list)  
    return random_list
```

- Pentru validarea răspunsului am următoarea funcție, care primește `data_list`, adică instanța problemei, `max_weight` și `random_list` pentru a verifica soluția generată, și returnează `-1` dacă soluția nu e bună, sau `current_value`, dacă soluția este bună.

```
4 usages  
def validate(data_list: list, max_weight: int, random_list: list):  
    current_weight = 0  
    current_value = 0  
    for i in range(len(random_list)):  
        if random_list[i] == 1:  
            # print(str(i)+" "+str(data_list[2][i]))  
            current_weight += data_list[2][i]  
            current_value += data_list[1][i]  
    # print(current_weight)  
    if current_weight <= max_weight:  
        return current_value  
    return -1
```

- Pentru a inițializa populația am o funcție care generează o întreagă populație cu valori random, inclusiv negative. Primește ca parametrii mărimea populației și cât de lungă să fie o soluție (`pop_size` respectiv `line_number`) și returnează o populație.

```
def population_initialization(pop_size, line_number):
    population = []
    for _ in range(pop_size):
        solution = generate(line_number)
        population.append(solution)
    return population
```

- Pentru a calcula valorile populatiei am o functie care primeste ca parametrii datele citite la inceput plus populatia, si returneaza lista de valori a respectivei populatii.

```
def population_value(data_list, max_weight, population):
    population_values = []
    for individual in population:
        individual_value = validate(data_list, max_weight, individual)
        population_values.append(individual_value)
    return population_values
```

- Pentru a alege un parinte am ales sa fac un turnir intre 5 indivizi cu valori valide, alesi la intamplare. Am o functie care primeste populatia, marimea turnirului si valoarea populatiei, si returneaza cel mai bun individ din turnir.

```
def tournament_selection(population, tournament_size, popvalue):
    # Filtrăm lista de indivizi care au fitness diferit de -1
    individuals_with_valid_fitness = [i for i, value in enumerate(popvalue) if value != -1]

    # Selectia aleatorie a indivizilor din lista filtrată
    selected_individuals = random.sample(individuals_with_valid_fitness, tournament_size)

    best_guy = None
    best_value = float('-inf') # Initializeaza cu un fitness negativ infinit
    for i in selected_individuals:
        if popvalue[i] > best_value:
            best_guy = population[i]
            best_value = popvalue[i]
    return best_guy
```

- Pentru incrucisare am decis sa fac 2 copii din 2 parinti, unde un copil se compune cu o parte de la un parinte pana la un punct de separare, apoi cu o parte de la celalalt parinte.

```
def crossover(parent1, parent2):
    # Selectăm un punct de tăiere aleator
    crossover_point = random.randint(a: 1, len(parent1) - 1)

    # Combinăm genele părinților pentru a crea descendente
    child1 = parent1[:crossover_point] + parent2[crossover_point:]
    child2 = parent2[:crossover_point] + parent1[crossover_point:]

    return child1, child2
```

- Pentru mutatie am ales sa fac o functie care sa primeasca un individ si o sansa de mutatie, iar sansa de mutatie se aplica pe fiecare gena a individului.

```
def mutation(individual, mutation_chance):
    mutated_individual = list(individual)
    for i in range(len(mutated_individual)):
        if random.randint(a: 1, b: 100) <= mutation_chance:
            # Se efectuează mutatia: inversarea valorii genei la un indice invers
            mutated_individual[i] = flip(mutated_individual[i])
    return mutated_individual
```

- In codul facut de mine, dupa mai multe rulari, exista multe valori de -1 din cauza mutatiilor indivizilor, valori pe care nu le consider valide. Am definit o functie care sa imi dea cea mai mica valoare obtinuta != -1

```
def mutation(individual, mutation_chance):
    mutated_individual = list(individual)
    for i in range(len(mutated_individual)):
        if random.randint(a: 1, b: 100) <= mutation_chance:
            # Se efectuează mutatia: inversarea valorii genei la un indice invers
            mutated_individual[i] = flip(mutated_individual[i])
    return mutated_individual
```

Cautare folosind algoritm evolutiv pentru problema rucsacului

- **Pas 1:** Citim datele de intrare din fisier:

```

def readTxt():
    index_list = []
    value_list = []
    weight_list = []

    with open('rucsac-20.txt', 'r') as f:
        line_number = int(f.readline().strip())
        for _ in range(line_number):
            line = f.readline().strip().split()
            numbers = [int(x) for x in line]
            index_list.append(numbers[0])
            value_list.append(numbers[1])
            weight_list.append(numbers[2])
        data_list = [index_list, value_list, weight_list]

        #         for i in range(line_number):
        #             print(data_list[0][i], data_list[1][i], data_list[2][i])

        max_weight = int(f.readline().strip())
        # print(max_weight)
    return line_number, data_list, max_weight

```

- Am decis sa fac o lista de liste, data_list, unde data_list[0] este indexul obiectului, data_list[1] este valoarea obiectului si data_list[2] este greutatea obiectului, iar fiecare obiect este accesat cu data_list[i][j], unde i={0,1,2} , j=0 pana la line_number.
- line_number este numarul de linii din fisier, data_list este lista creata pentru prelucrare – instanta si max_weight este greutatea maxima admisa.
-
- **Pas 2:** Initializam populatia, calculam valoarea ei

```

def evo_alg(pop_size, num_gen, mutation_chance):
    start_time = time.time()
    line_number, data_list, max_weight = readTxt()
    population = population_initialization(pop_size, line_number)
    popvalue = population_value(data_list, max_weight, population)

```

- **Pas 3:** Pentru fiecare generatie, facem urmatoarele:

```

for _ in range(num_gen):

```

- **Pas 3.1:** Cautam parinti

```

for _ in range(num_gen):
    selected_parents = []
    # Selectarea părintilor
    for _ in range(pop_size // 2):
        parent = tournament_selection(population, tournament_size: 5, popvalue)
        selected_parents.append(parent)

```

- **Pas 3.2:** Cream noua generatie folosind parintii gasiti, generatie este formata prin incrucisare si apoi prin mutatia genelor.

```

# Reproducerea (incrucisare si mutatie)
new_generation = []
for i in range(len(selected_parents)):
    parent1 = selected_parents[i]
    parent2 = selected_parents[(i + 1) % len(
        selected_parents)] # Utilizăm operația modulo pentru a accesa primul parinte în cazul ultimului parinte
    child1, child2 = crossover(parent1, parent2)
    child1 = mutation(child1, mutation_chance)
    child2 = mutation(child2, mutation_chance)
    new_generation.append(child1)
    new_generation.append(child2)
# print('nr indivizi noua gen', len(new_generation))

```

- **Pas 3.3:** Evaluam noua populatie, si o ordonam descrescator in functie de cea mai buna valoare.

```

# Evaluarea și selectia noii generatii
popvalue = population_value(data_list, max_weight, new_generation)
for i in range(len(popvalue) - 1):
    for j in range(i + 1, len(popvalue)):
        if popvalue[i] < popvalue[j]:
            popvalue[i], popvalue[j] = popvalue[j], popvalue[i]
            new_generation[i], new_generation[j] = new_generation[j], new_generation[i]
    population = new_generation.copy()
# print(len(popvalue))

```

- **(Pentru rezolvarea laboratorului)** Salvez in fisierul rez1.txt urmatoarele informatii: marime populatie, numar generatii, sansa mutatie genetica, best, avg, worst si timpul de rulare.

```

sum = 0
for i in range(len(popvalue)):
    sum += popvalue[i]

worst = get_smallest_non_negative(popvalue)
end_time = time.time()
runtime = end_time - start_time
with open("rez1.txt", "a") as file:
    file.write("Marime populatie: " + str(pop_size) + "\n")
    file.write("Numar generatii: " + str(num_gen) + "\n")
    file.write("Sansa mutatie genetica: " + str(mutation_chance) + "\n")
    file.write("Best: " + str(popvalue[0]) + "\n")
    file.write("Avg: " + str(sum / len(popvalue)) + "\n")
    file.write("Worst: " + str(worst) + "\n")
    file.write("Timpul de rulare: " + str(runtime) + "\n\n")
return popvalue

```

Problema Comis-Voiajorului

- Se consideră n orașe și un comis-voiajor care trebuie să viziteze toate orașele, trecând o singură dată prin fiecare și să se întoarcă în orașul inițial astfel încât costul total să fie cât mai mic.
- Soluția poate fi reprezentată ca o permutare de n orașe. Costul unei soluții este suma distanțelor dintre orașe în ordinea în care apar în permutare.
- Pentru generarea unei soluții aleatoare am o funcție cu un parametru, o lista de noduri, și returnează lista de noduri cu pozițiile lor schimbate random:

```

def generate_random_chromosome(nodes):
    chromosome = list(nodes)
    random.shuffle(chromosome)
    return chromosome

```

- Pentru a calcula distanța dintre 2 puncte folosesc o formulă simplă într-o funcție care are 3 parametri, primul este lista de noduri, și al doilea și al treilea parametru sunt nodurile între care mă interesează distanțele:

```
def calculate_distance(coords, node1, node2):
    x1, y1 = coords[node1]
    x2, y2 = coords[node2]
    return np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
```

- Pentru a evalua o solutie, am o functie care primeste o solutie si nodurile problemei, si returneaza distanta solutiei.

```
def evaluate_chromosome(chromosome, coords):
    dist = 0
    for i in range(len(chromosome)):
        dist += calculate_distance(coords, chromosome[i - 1], chromosome[i])
    return dist
```

- Pentru a genera populatia initiala am o functie care primeste nodurile problemei si o marime de populatie, si returneaza populatia generata aleator

```
def generate_initial_population(nodes, population_size):
    population = []
    for _ in range(population_size):
        chromosome = generate_random_chromosome(nodes)
        population.append(chromosome)
    return population
```

- Am facut o functie care returneaza toate distantele unei populatii

```
def evaluate_population(population, coords):
    distances = []
    for chromosome in population:
        distance = evaluate_chromosome(chromosome, coords)
        distances.append(distance)
    return distances
```

- Pentru selectarea parintilor am facut o functie care ia drept parinti indivizii cu cel mai bun traseu(cele mai scurte) si returneaza o lista de parinti

```
def selection(population, distances):
    selected_parents = []
    for _ in range(len(population)):
        # Selectează un părinte în funcție de distanța asociată
        index = distances.index(min(distances))
        selected_parents.append(population[index])
        # Setează distanța selectată ca infinit pentru a nu fi selectată din nou
        distances[index] = float('inf')
    return selected_parents
```

- Pentru încrucișare am o funcție care are 2 părinți și creează 2 copii, cu un punct de tăiere al părinților ales la întâmplare

```
def crossovertsp(parents):
    offsprings = []
    for parent1, parent2 in zip(parents[:2], parents[1:2]): # Iterăm prin părinții în perechi (de la 0 la lungimea listei, cu pasul 2)
        # Aplicăm crossover
        crossover_point = random.randint(a=0, b=len(parent1))
        child1 = parent1[:crossover_point] + parent2[crossover_point:]
        child2 = parent2[:crossover_point] + parent1[crossover_point:]
        offsprings.extend([child1, child2])
    return offsprings
```

- Pentru mutație am o funcție care primește o populație și șansa de mutație, dacă se întâmplă o mutație a unui individ, o să se facă swap la 2 locuri în individ. Se returnează lista de indivizi (tehnice noua populație)

```
def mutationtsp(offsprings, mutation_chance):
    for offspring in offsprings:
        if random.randint(a=1, b=100) <= mutation_chance:
            # Aplică o mutație asupra cromozomului (de exemplu, schimbând două orașe în rută)
            index1, index2 = random.sample(range(len(offspring)), k=2)
            offspring[index1], offspring[index2] = offspring[index2], offspring[index1]
    return offsprings
```

- Mai am o funcție care doar actualizează noua populație

```
def update_population(offsprings):
    # Înlocuiește populația veche cu noua populație (offsprings)
    return offsprings
```

Cautare folosind algoritm evolutiv pentru problema Comis-Voiajorului

- **Pas 1:** Încarcăm datele problemei, generăm prima populație și calculăm distanțele ei

```
def evo_alg_tsp(pop_size, num_gen, mutation_chance):
    start_time = time.time()
    problem = tsplib95.load_problem("eil76.tsp")
    best_solution = None
    best_distance = float('inf')
    all_distances = []

    nodes = list(problem.get_nodes())
    coords = problem.node_coords

    population = generate_initial_population(nodes, pop_size)
    distances = evaluate_population(population, coords)
```

- **Pas 2:** Pentru fiecare generatie, facem urmatoarele:

```
for _ in range(num_gen):
```

- **Pas 2.1:** Cautam parinti

```
parents = selection(population, distances)
```

- **Pas 2.2:** Cream noua generatie

```
offsprings = crossovertsp(parents)
offsprings = mutationtsp(offsprings, mutation_chance)
```

- **Pas 2.3:** Pentru noua generatie, aflam cea mai buna distanta

```
best_solution_index = distances.index(min(distances))
current_solution = population[best_solution_index]
current_distance = min(distances)

if current_distance < best_distance:
    best_solution = current_solution
    best_distance = current_distance
    #print(best_distance)
```

- **(Pentru rezolvarea laboratorului)** Salvez in fisierul rez2.txt urmatoarele informatii: marime populatie, numar generatii, sansa mutatie genetica, best, avg si timpul de rulare.

-

```

avg_distance = sum(all_distances) / len(all_distances)
end_time = time.time()
runtime = end_time - start_time

with open("rez2.txt", "a") as file:
    file.write("Marime populatie: " + str(pop_size) + "\n")
    file.write("Numar generatii: " + str(num_gen) + "\n")
    file.write("Sansa mutatie genetica: " + str(mutation_chance) + "\n")
    file.write("Best: " + str(best_distance) + "\n")
    file.write("Avg: " + str(avg_distance) + "\n")
    file.write("Timpul de rulare: " + str(runtime) + "\n\n")

return best_distance, avg_distance

```

Tabele de date

- Pentru cautare folosind algoritm evolutiv la problema rucsacului:

Instanta problemei	Marime populatie	Numar generatii	Sansa mutatie genetica	Numar executari	Best value	Average value	Worst value	Avg. time
rucsac20.txt	100	100	10	10	726	391.818	326	0.12803927516937257
rucsac20.txt	100	500	10	10	726	391.069	333	0.6415812215099335
rucsac20.txt	100	1000	10	10	726	380.372	354	1.2532087564468384
rucsac20.txt	250	100	10	10	726	368.8572	225	0.4403317451477051
rucsac20.txt	250	500	10	10	726	372.0704	349	2.1947
rucsac20.txt	250	1000	10	10	726	371.8476	267	4.4848
rucsac20.txt	500	100	10	10	726	375.0078	276	1.4222
rucsac20.txt	500	500	10	10	726	382.7228	263	7.0769
rucsac20.txt	500	1000	10	10	726	375.3332	232	13.9847
rucsac200.txt	100	100	10	10	133041	71338.473	105241	0.7785
rucsac200.txt	100	500	10	10	132778	72595.318	105567	3.9612
rucsac200.txt	100	1000	10	10	133312	74541.471	99782	7.7608
rucsac200.txt	250	100	10	10	132942	70810.331	103828	2.08365
rucsac200.txt	250	500	10	10	133232	71887.334	95185	10.42461
rucsac200.txt	250	1000	10	10	133030	71430.154	94969	20.35523
rucsac200.txt	500	100	10	10	133493	71113.94	98614	4.763718
rucsac200.txt	500	500	10	10	133085	73104.732	100206	23.380005
rucsac200.txt	500	1000	10	10	133090	72298.273	101846	48.371673

- Pentru problema TSP folosind Taboo list:

Instanta problemei	Marime populatie	Numar generatii	Sansa mutatie genetica	Numar executari	Best distance	Average distance	Avg. time
eli76.tsp	100	100	10	10	1367.3275	1696.1937	0.8959
eli76.tsp	100	250	10	10	1172.2247942254216	1539.4854	2.1023
eli76.tsp	100	500	10	10	1011.4072	1413.1425	4.4855
eli76.tsp	250	100	10	10	1239.0384	1602.2652	2.2927
eli76.tsp	250	250	10	10	1056.1417	1420.1124	5.6338
eli76.tsp	250	500	10	10	934.7114	1262.8936	11.3139
eli76.tsp	500	100	10	10	1174.8317	1538.4895	4.8928
eli76.tsp	500	250	10	10	957.3104	1327.7515	11.7785
eli76.tsp	500	500	10	10	756.3279	1162.1352	23.6757

Observatii

- La problema rucsacului un parametru care pare bun de crescut este populatia, pe cand la problema TSP parametrul care pare mai potrivit de crescut este numarul de generatii
- Toate datele de rulare sunt facute cu o rata de mutatie de 10%.
- Pentru problema TSP am folosit un pachet din python specializat pentru problemele de acest gen, care are cateva functii definite pentru citire, afisare, prelucrare de date

- Rezultatele rularilor pentru problema 1 (Cautare folosind algoritm evolutiv pentru problema rucsacului) se afla in rez1.txt, iar rezultatele pentru problema 2 (Cautare folosind algoritm evolutiv pentru problema Comis-Voiajorului) se afla in rez2.txt

```
import tsplib95
```

Concluzii

- Algoritmul evolutiv are rezultate vizibil mult mai bune din toate punctele de vedere: medii mult mai bune, maxime mai mari, rezultate mai bune pentru timpii de rulare.